

Chapter 3: Operating Systems: Concurrency & Synchronization

3.1 Processes, Threads, and Race Conditions

Shared Memory and Non-Deterministic Interleaving

While a process encapsulates an isolated virtual address space, threads within the same process share code, global data, and open file descriptors while maintaining private program counters, registers, and call stacks.

When multiple threads concurrently read and write shared data without synchronization, a Race Condition occurs: the final outcome depends unpredictably on the arbitrary interleaving of thread execution scheduled by the OS.

3.2 The Critical Section Problem & Semaphores

Mutual Exclusion, Progress, and Bounded Waiting

A critical section is a block of code accessing shared resources that must not be concurrently executed by more than one thread.

A valid synchronization primitive must satisfy: 1. Mutual Exclusion: At most one thread in the critical section. 2. Progress: If no thread is in critical section, waiting threads must not be blocked indefinitely. 3. Bounded Waiting: A limit exists on the number of times other threads can enter before a requesting thread is served.

Edsger Dijkstra introduced Semaphores: integer variables manipulated exclusively through atomic operations `wait()` (P) and `signal()` (V).

3.3 Deadlocks and Coffman's Four Conditions

System Stagnation and Prevention

A deadlock is a condition where a set of threads are blocked because each holds a resource and waits for another resource held by another thread in the set.

Deadlock can arise if and only if all four Coffman conditions hold simultaneously: 1. Mutual Exclusion: At least one non-shareable resource. 2. Hold and Wait: A thread holding at least one resource is waiting for additional resources. 3. No Preemption: Resources cannot be forcibly seized; only voluntarily released. 4. Circular Wait: A closed chain of threads exists where each waits for a resource held by the next.

Key Takeaways:

- Breaking any one of Coffman's four conditions guarantees deadlock prevention.
- Mutex locks provide binary mutual exclusion; counting semaphores coordinate resource pools.
- Priority inversion occurs when a lower-priority thread holds a lock required by a high-priority thread.